

Developer Operations (devops)

Operational Basics

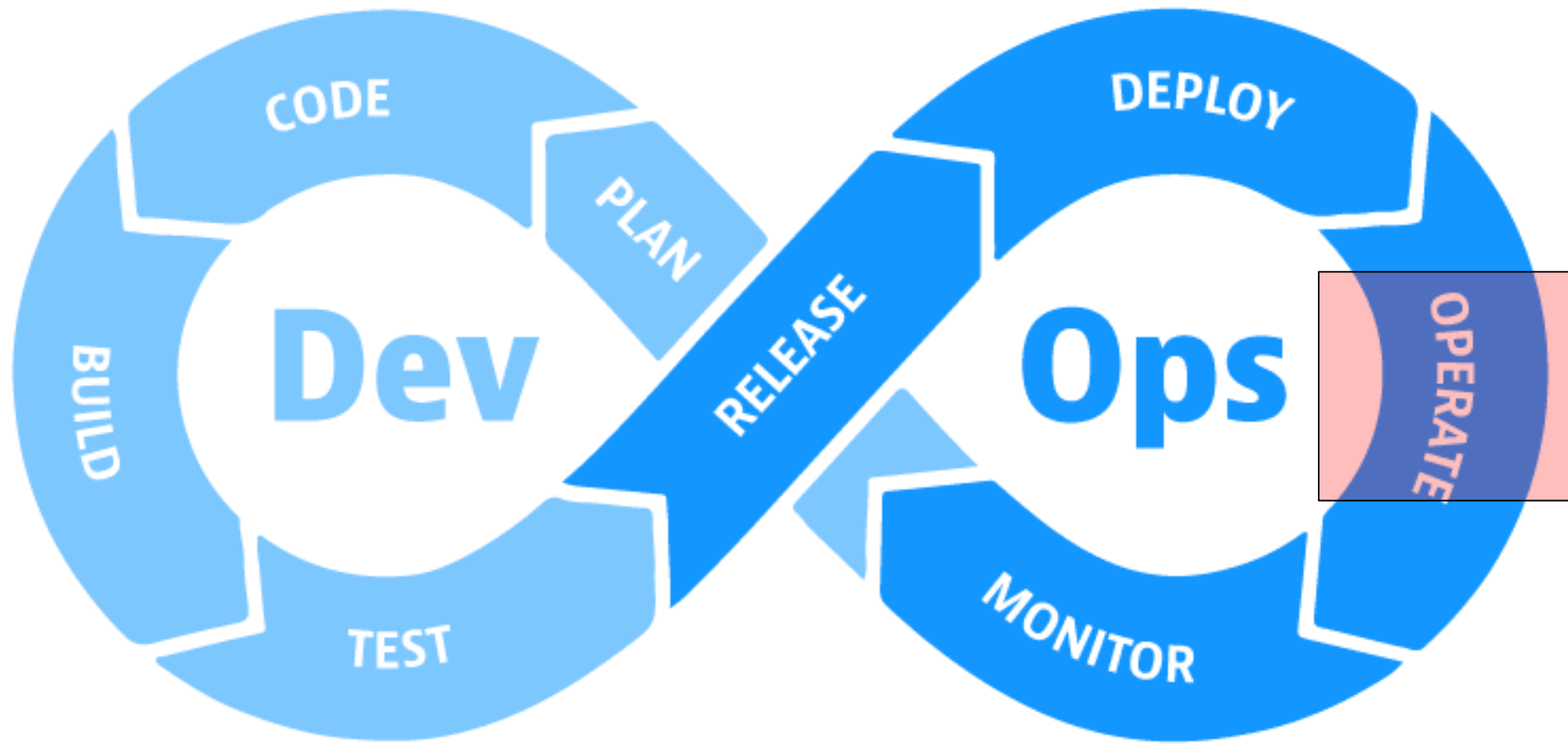


Citation of the day

*—If Tetris has taught me anything, it's that errors pile up and
accomplishments disappear*

—Andrew Clay Shafer

Basics of Operating



Basics of Operations

How do you handle deploys?

- Must be tested
- Trigger:
 - Hotfix
 - Zeitlich mittels ITSM / Wartungsfenster
- Direkt auf FTP -> Prod

How do you “operate”
(handle the time between deploys)?

- Features Umsetzen für Dev/Test
- Lead Time, 3 Monate

Basics of Operations, IT System Management

Governance:

- Service Catalog
- Knowledge Base
- Asset Management

Changes / Deployments:

- Asset Management
- Change Management
- Problem Management
- Incident Management

Operations by Process:

- Incident Management
- Service Desk
- Service Catalogment



Paradigm Switch to Products

Governance:

- Service Catalog
 - Front-Up building of Services
 - Manually managed
 - Out of date
- Knowledge Base
 - Manually written articles
 - Often not necessarily related to incidents
- Asset Management
 - List of Software, Server, Networks, Releases
 - Manually maintained

Product View of Technology:

- **Building Blocks and API Mgmt**
 - Organically build up services
 - Automatically accessible via API Mgmt
 - Always up to date
- **Innersource and Open Sourcecode (internally)**
 - Automatically generated articles
 - Wiki
- **Monitoring and Reporting**
 - List of Software, Networks, Server, Releases
 - Automatically determined

Paradigm Switch to “release often, release early”

Changes / Deployments:

- Asset Management
 - List of Software, Server, Networks, Releases
 - Manually maintained
- Change Management
 - Centralized Management of Changes, Releases
 - Approve/ Deny if process not held
- Problem Management
 - Problem Ticket, Discussion, Contracts
 - Person/Role-based

Continuous Deployment

- *Monitoring and Reporting*
 - List of Software, Networks, Server, Releases
 - Automatically determined
- *Continuous, Automatic Deployment*
 - Automatic, non-blocking deployment to prod
 - Not Possible, if not deployed via pipeline
- *Blameless culture*
 - PostMortem: all humans make failures
 - Team-Based
- *Incident Solving*
 - Product driven

Paradigm Switch to “You built it, you run it”

Operations by Process:

- Incident Management
 - Workflow driven
 - Role-based
 - Wrapped around all products
- Service Desk
 - Centralized Support
 - Wrapped around all products
- Service Cataloge
 - List of Service, Software Releases, etc.
 - Manually maintained

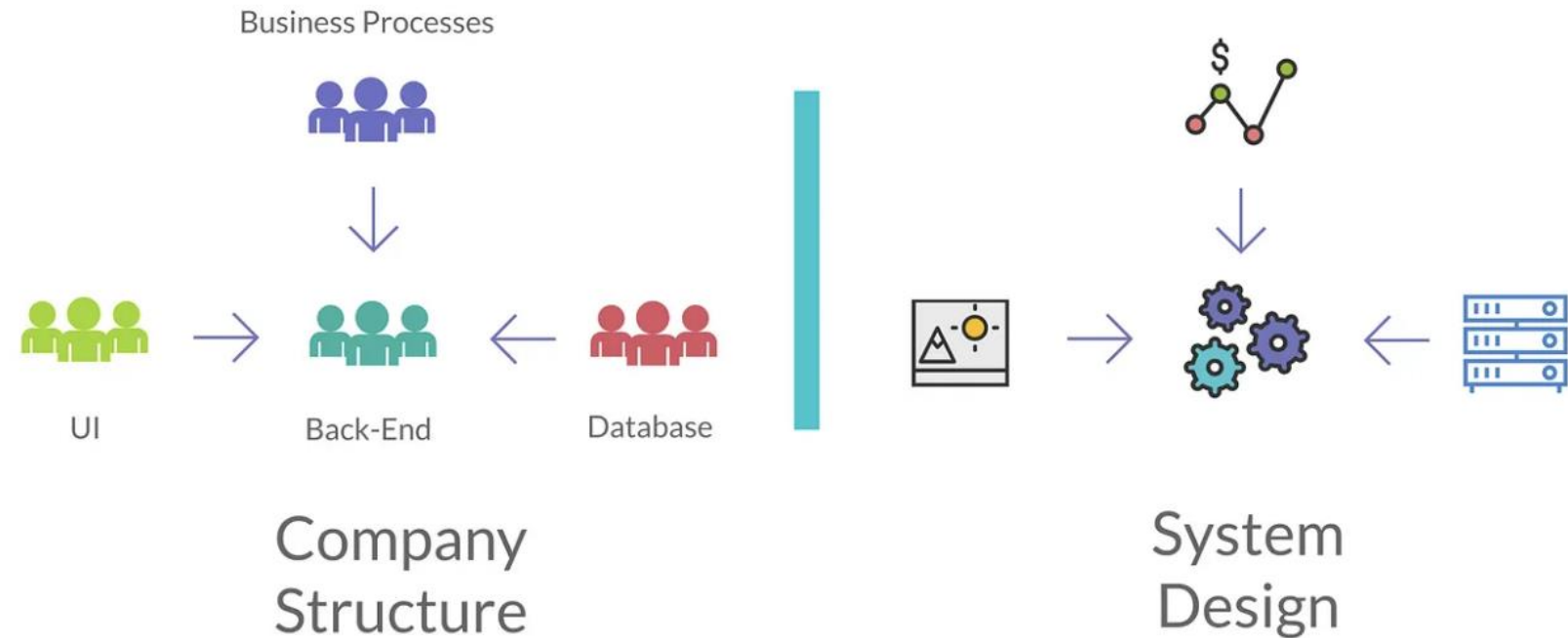
Operations by DevOps

- Incident Solving
 - Product driven
 - Team-based
 - Part of products
- Internal Support
 - 1st and 2nd level: Done by automatisms / docum.
 - 3rd level: Team itself
 - List of Service, Software Releases, etc.
 - Automatically determined

Recap: Conways Law

“Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure”

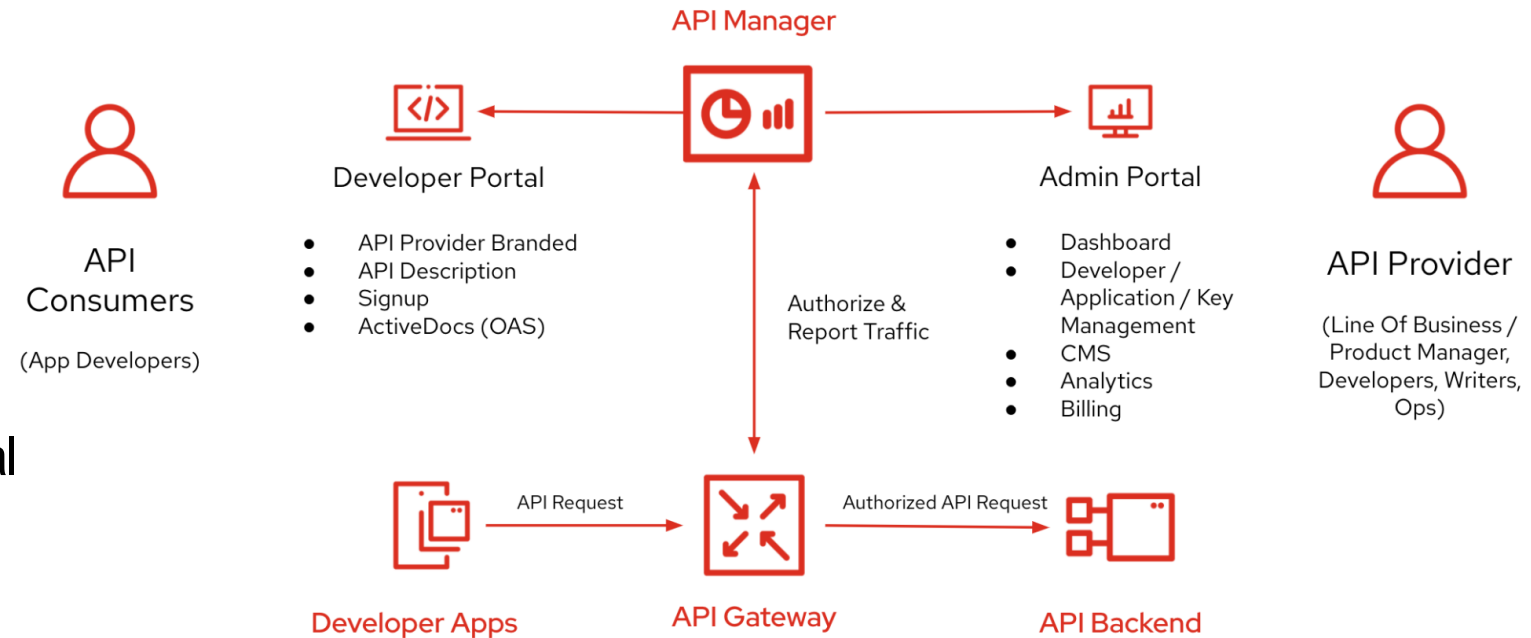
- Melvin E. Conway



API-Management

All API-Accesses must be provided through a centralized portal:

- Clear (horizontal) separation of concerns
 - Who owns which application?
- Protection of APIs against Brute Force, Internal Denial of Service “Usages”
- Versioning of APIs becomes necessary
- Automatism necessary
 - For Signup
 - For registration
 - Fits best with Gitops



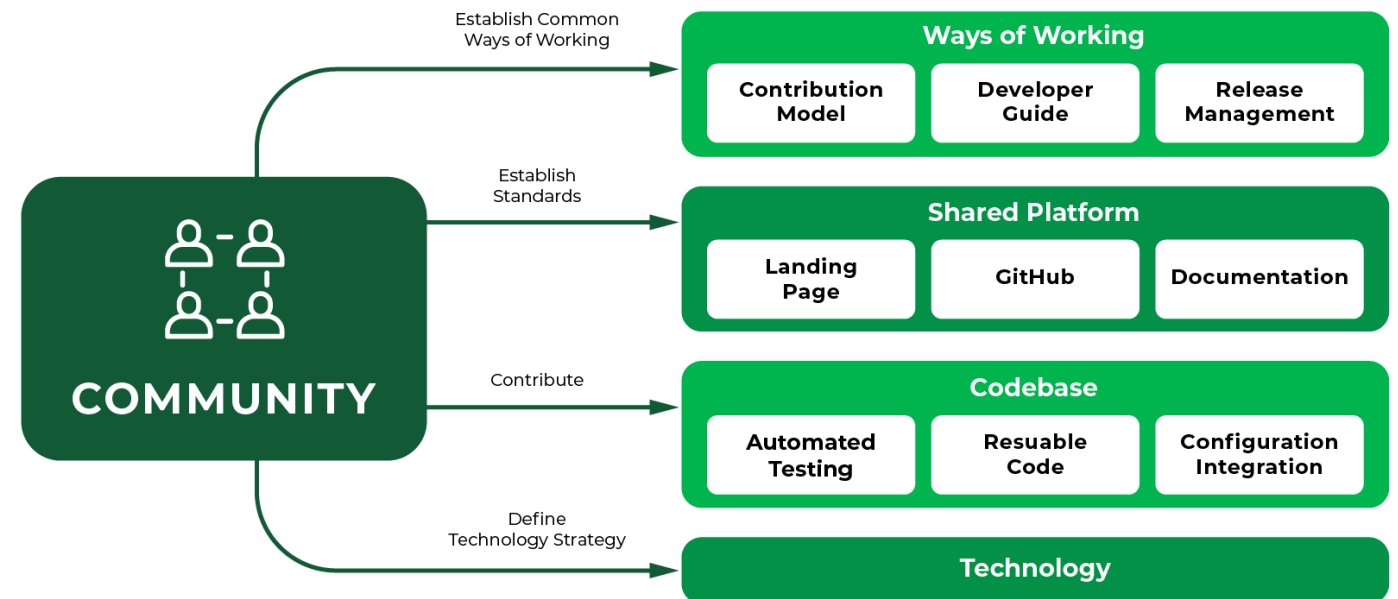
Innersource and Shared Responsibility

Commonly used Software should be owned by one team but maintained by multiple teams:

- Monitoring Dashboards
- CI/CD-Pipelines

Maintaining together defines de facto standards:

- Open Source for internal usage
- Enforces high quality standards



Monitoring and Observability

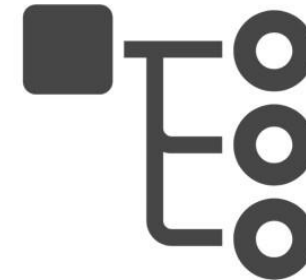
Observability should be open, transparent and be manageable via sourcecode:

- Centralized / federated Infrastructure
- Transparent, even to internal users / customers
- Easy to maintain
 - Label-based
 - Scalable (by adding more resources)
 - Extensible (e.g. by adding new dashboards / functionality)

Three pillars of observability



Metrics



Traces



Logs

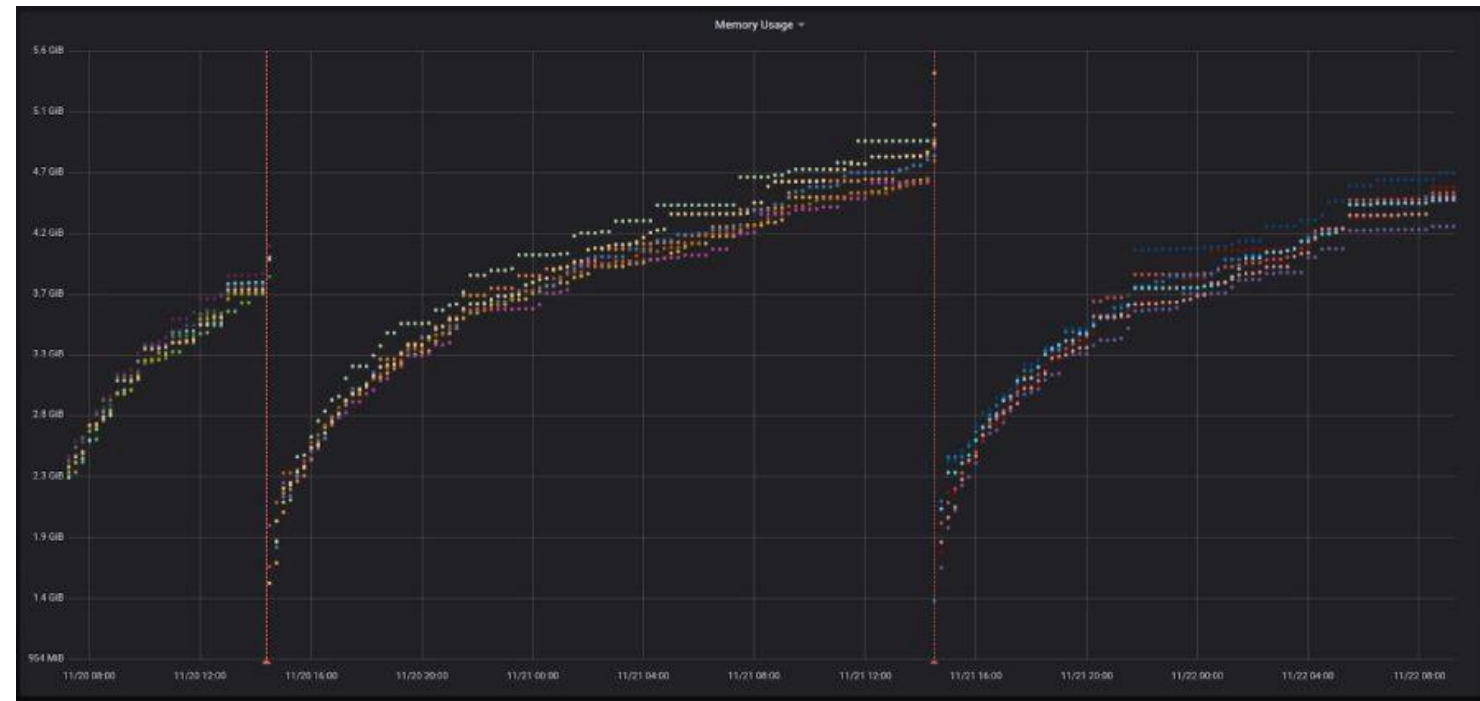
Changemgmt vs Changemonitoring

Changes apply too fast to be planned

- (semi) automatic
- Partially not intended
(complex architecture plus stacked infrastructure)

Solution?

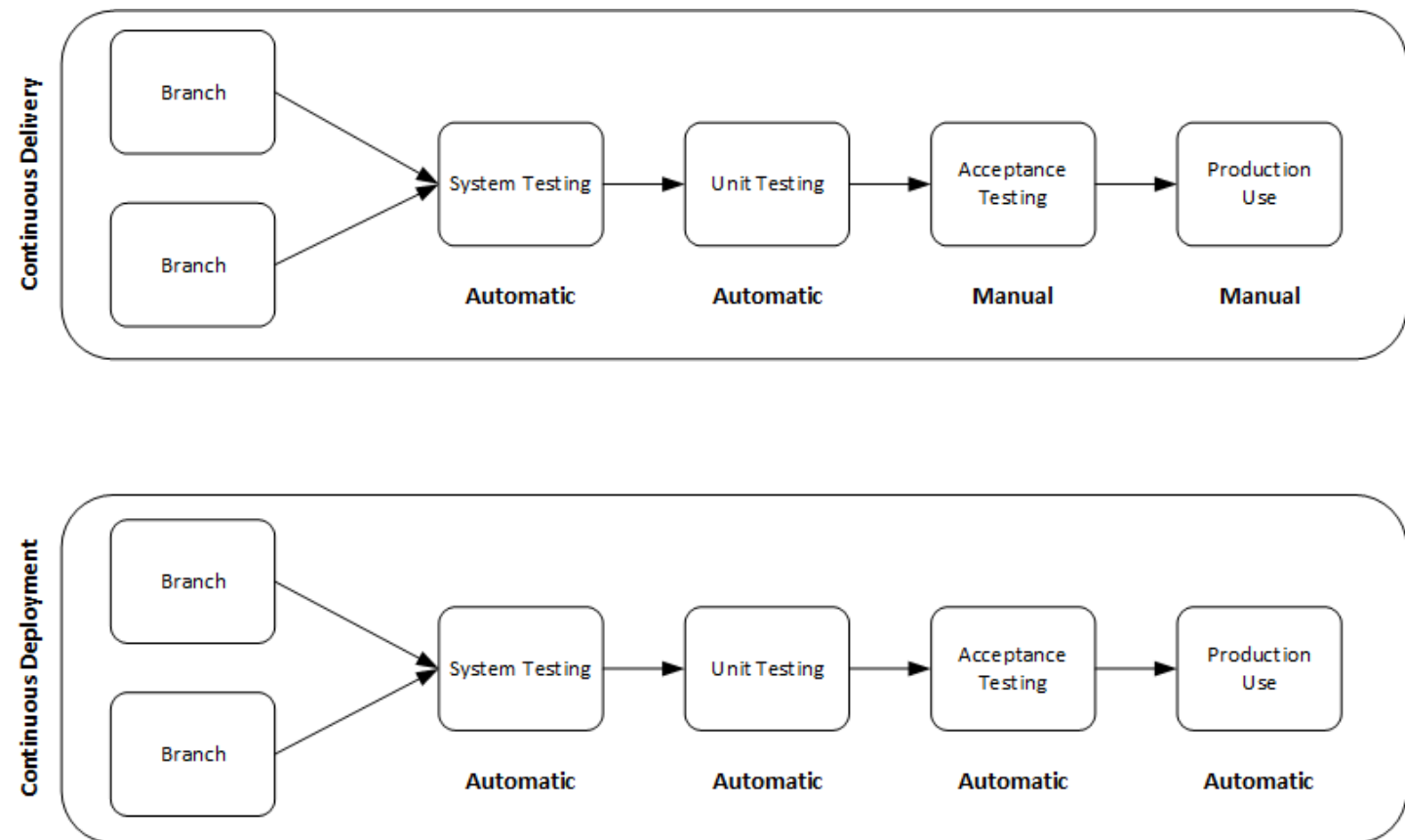
- Monitoring must annotate occurring changes to make impact visible



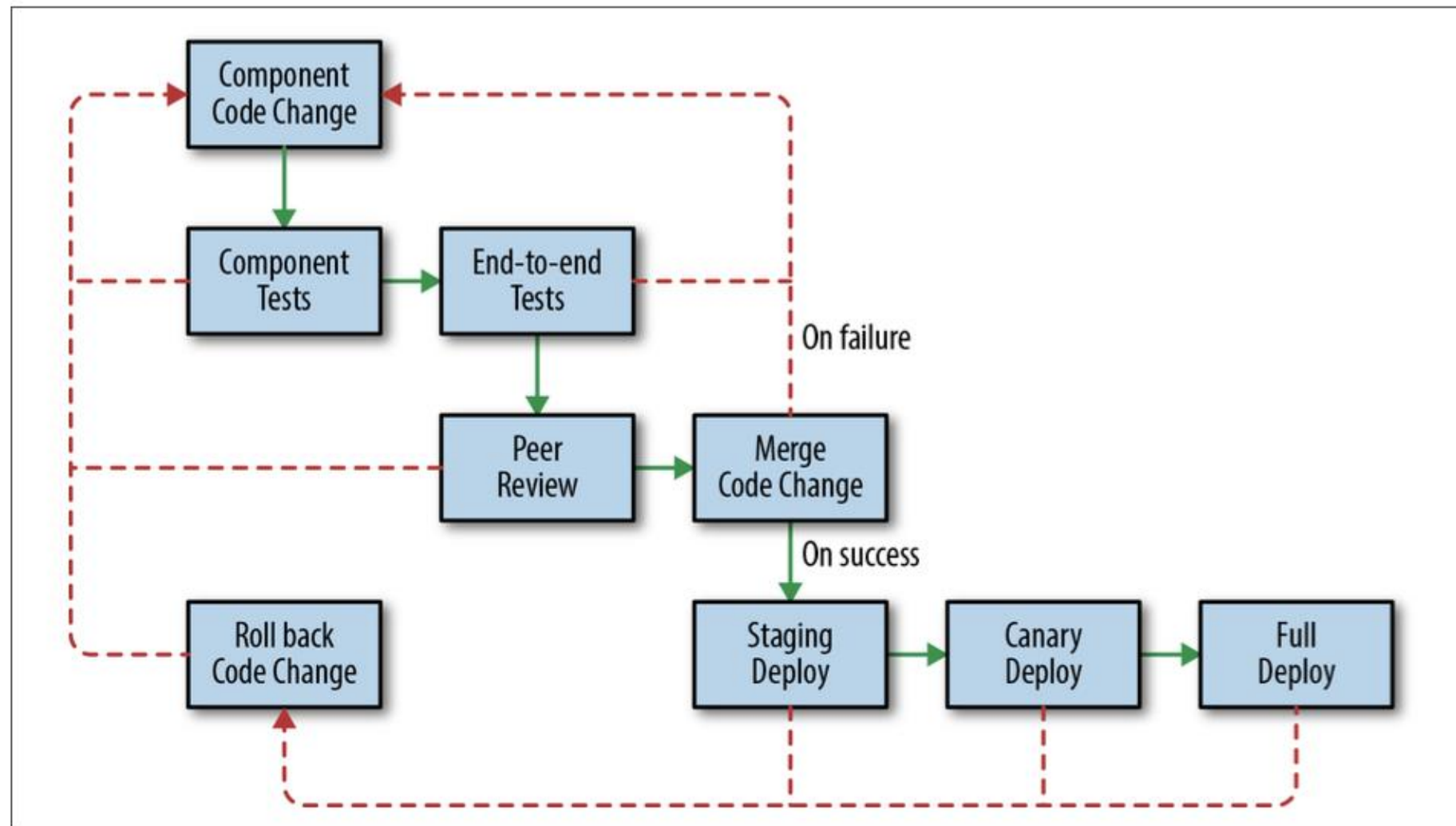
Continuous, Automatic Deployment

Fully automated workflows deploy on different stages

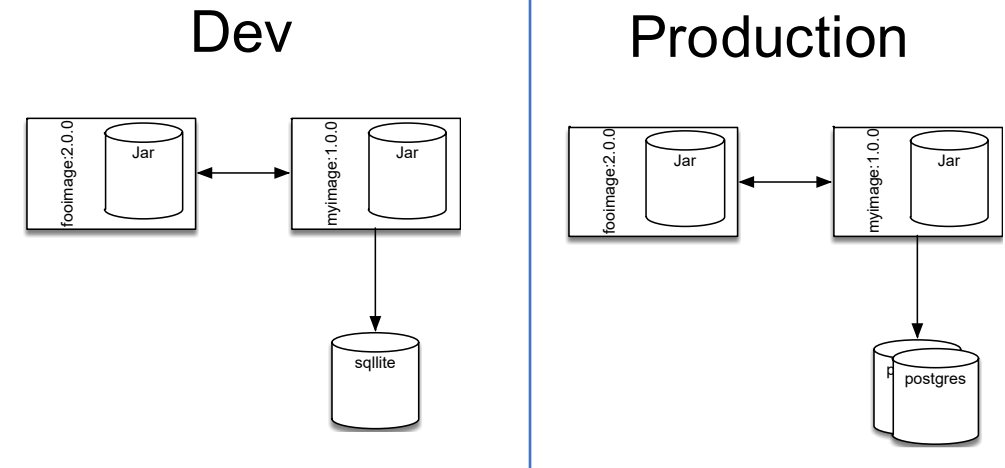
- Gitops may apply
- Common Toolset for all employees
- All automatic (or all manual)
- Manual approval may apply
- Stages: Different environments for different purposes



Deploys might be part of a bigger game...

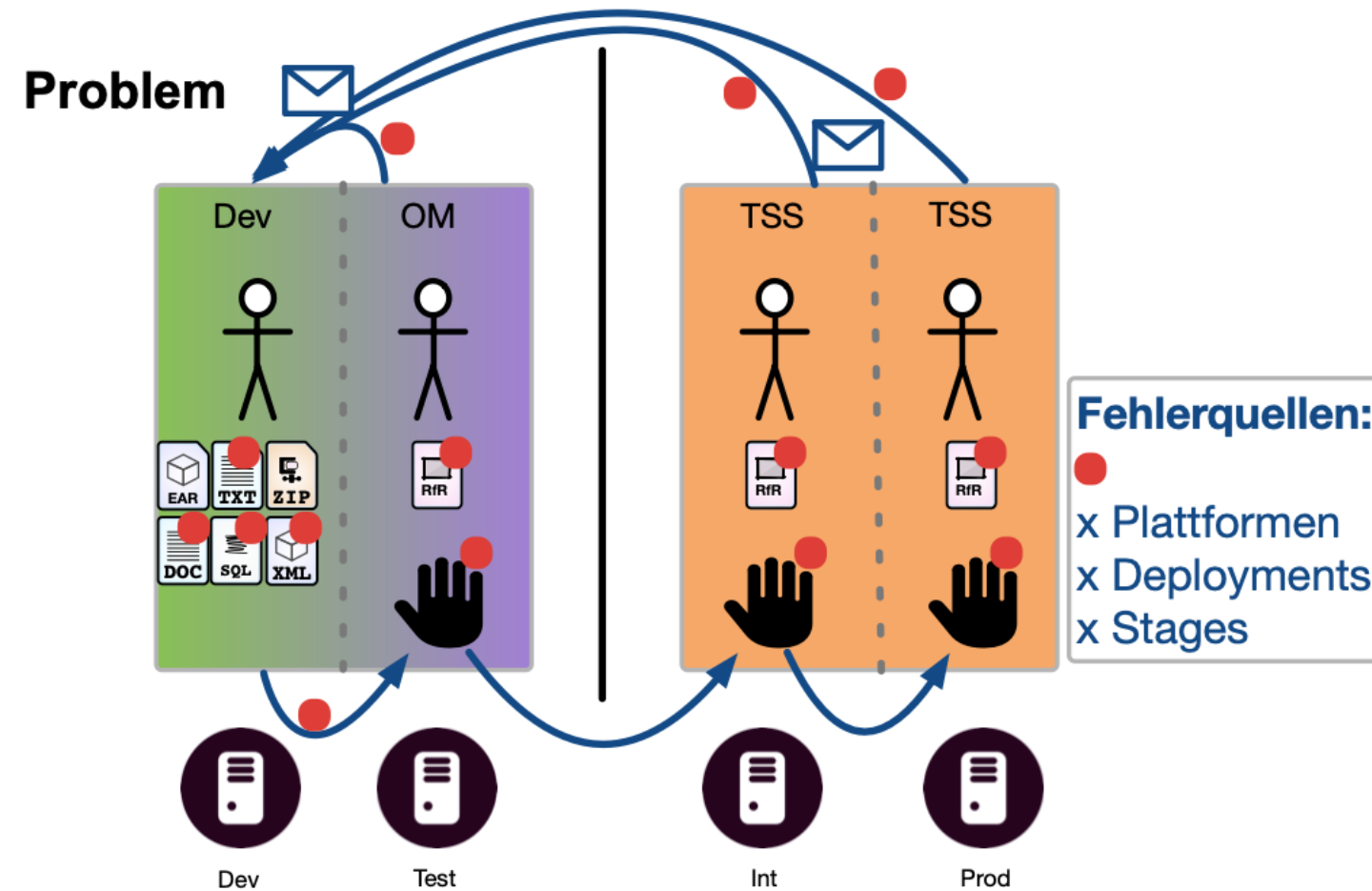


Recap: Dev/Prod Parity, 12factor



Criteria	Old World	New World
Time Gab	Teams develop, staging takes place, ending with Prod-Deployment → Took Weeks	Teams develop, Automatism push to production → should take hours
Personal Gab	Devs develop, Ops deploy	Team develop and deploys
Tool Gab	Devs have other tools than Ops	Common toolset for all stages

Recap: Websphere Deployment



Dev:

- Build: Maven and Nexus
- Deploy: Jenkins and own Maven-Plugin or SSH
- Documentation: No Deploymentdocumentation, all “automated”
- Database: Flyway

Ops:

- Build: Download ins. Websphere-Stübli: Bash in Maven-Archetype
- Deploy: Jython-Scripts
- Documentation: Word
- Database: out of Flyway generated SQL

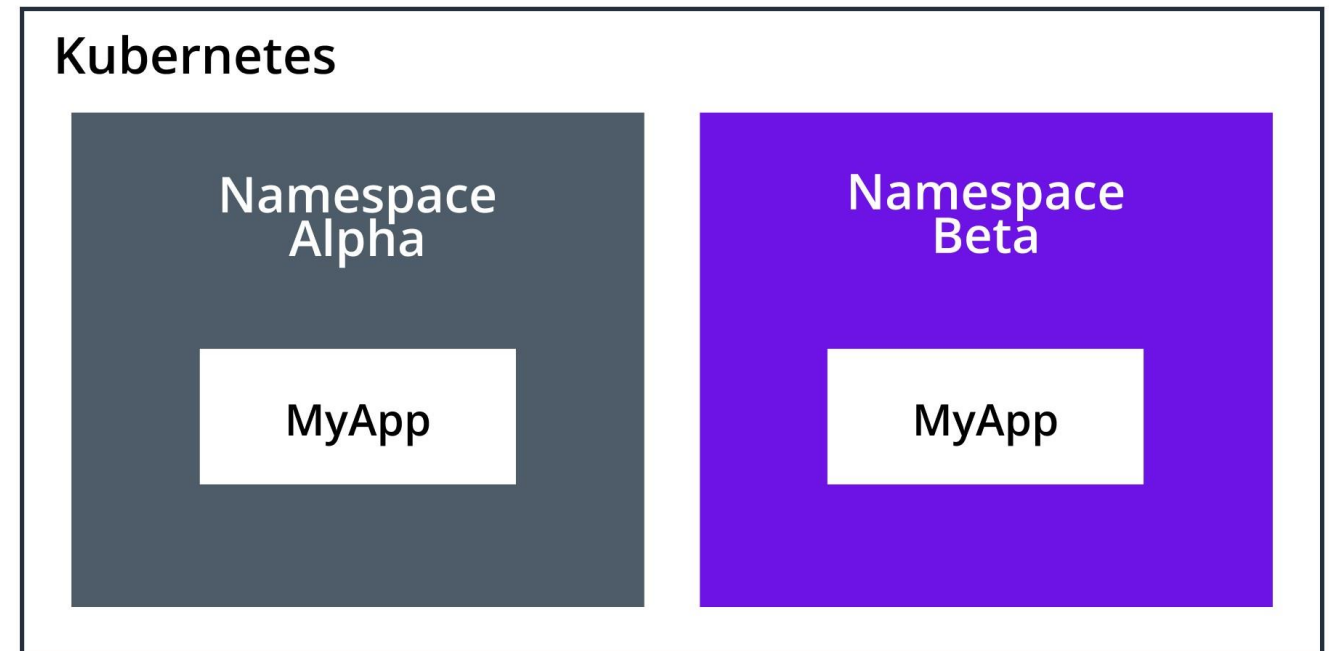
Staging and Namespaces

Use the namespacing-concept for implementing different stages:

- Stages have same network / infrastructure environment
- Stages are accessible in a similar way

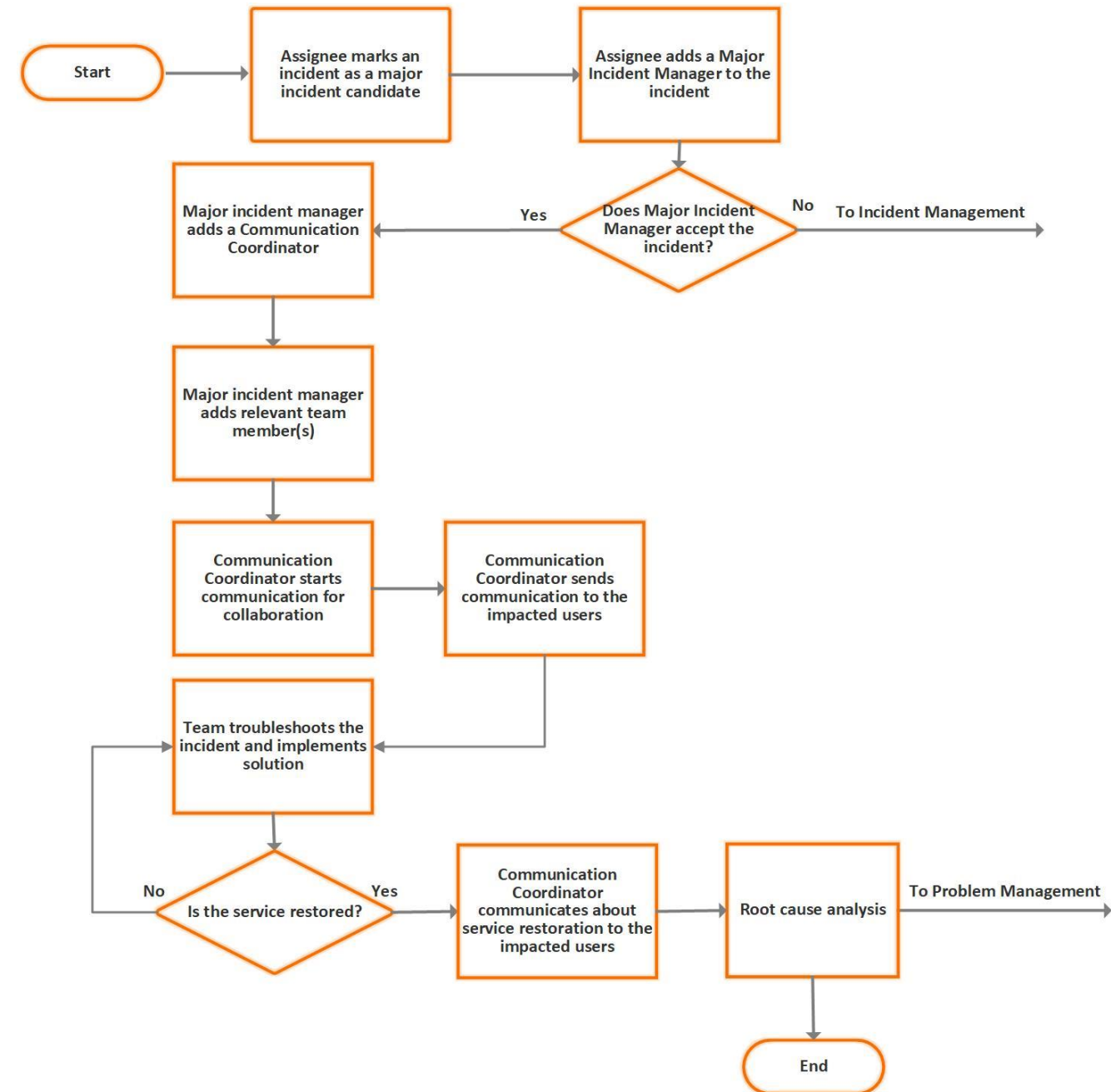
Nevertheless:

- Stages are isolated regarding the access, tokens, users, service accounts, resources.



Incident Management by ITIL

- Input varies, workflow stays static:
One Workflow for all services
- If not major, incidents may get stuck
 - Hard to determine correlations
- If major, hot potato game starts
 - Supplies
 - Internal Teams
 - Operations
- After major, hot potato game continues
 - intransparency



Incidents managed by DevOps

- Incidents are handled tool centric / team centric
The workflow must fit the team, the availability must be hold
- Establish structures for services beforehand
 - Warroom
 - Communication Channels
 - Internally
 - externally
 - Rely on each other → all hands on deck
- If you have a major, you will recognize it
- Be transparent afterwards, identify technical debt and increase availability

Summary

- No more silos
- Incidents are normal
- Changes should be granular
- Tooling and Culture are interrelated
- Measurement is crucial

